

Offline Secure Password & Note Vault

A command-line tool to store passwords, lock combos, and notes --
encrypted

Project Report

Course: Python Essentials

2. Introduction

Everyone has a bunch of passwords, PINs, and random stuff they need to remember -- Wi-Fi passwords, locker combos, private notes, whatever. Most people either reuse the same weak password everywhere or just dump it all in their phone's Notes app, which isn't really safe. Cloud password managers exist, but you're basically trusting some company's server with all your secrets, and you need internet to use them.

So for this project I built the opposite of that: a **fully offline** command-line tool that encrypts everything and saves it to one file on your own computer. Nothing gets sent anywhere. You unlock it with one master password, and that password is never saved anywhere -- only a hash of it is kept, just to check if you typed it right.

This was made for the **Python Essentials** course as my individual project. I tried to actually use stuff from the course (functions, modules, error handling, working with a third-party library) instead of just throwing code together, and also hit the requirements: CLI only, at least 3 functional modules, at least 4 non-functional requirements, and 5-10 files.

Below I go through the problem I was solving, how I designed it, how I built it, how I tested it, and what I learned doing it.

3. Problem Statement

3.1 Background

Over time everyone collects a huge pile of passwords, PINs, and notes they need to keep private. Writing them in plain text somewhere (sticky note, unencrypted file, default Notes app) is common but not safe at all -- anyone with access to your device can just read it. Cloud password managers fix the convenience problem but now you're trusting a company with your data and you need internet access to get to it.

3.2 The Problem

Basically, I wanted something that: never sends your data anywhere over the internet; actually uses real encryption instead of some basic trick like reversing the string or XOR-ing it; runs from the terminal with no GUI or server needed; and doesn't break or corrupt your data if your laptop crashes or loses power while it's saving.

3.3 Scope

What I built: local encrypted storage for three types of entries (passwords, lock combos, notes), a master password that unlocks everything, add/view/edit/delete/search on entries, a password generator, tags, masked input so your password doesn't show on screen, and clipboard copy.

What I didn't build: no cloud sync between devices, no browser autofill, no multiple users sharing one vault, and no way to recover your master password if you forget it (that's on purpose -- if there was a recovery option it would mean there's a backdoor). Also no GUI since the assignment says it has to run from the command line.

3.4 Target Users

- Basically anyone who wants a personal password vault without trusting it to some company's cloud.
- Students (like me) who want to actually see key derivation and encryption working in real code, not just read about it.
- Anyone who wants an offline backup option for their sensitive info that isn't dependent on the internet or a company staying in business.

4. Functional Requirements

I split the project into three main functional modules, each one doing its own job, which covers the course's requirement of at least 3 functional modules.

#	Module	What it does	File
1	Authentication	Sets up and checks the master password using a salted, hashed value. Never actually saves the password anywhere.	auth.py
2	Encryption / Decryption	Turns the master password into an encryption key and uses it to lock/unlock the whole vault's data.	crypto_engine.py
3	Entry Management (CRUD)	Add, list, search, view, edit, and delete entries -- passwords, lock combos, or notes -- each with optional tags.	vault_manager.py

How input/output works

- **Input:** command-line arguments (using argparse) plus a separate masked prompt for anything secret, so passwords never end up sitting in your shell history.
- **Output:** clean tables for listing entries, plain confirmation or error messages for everything else.
- **How a command runs:** every command does the same basic thing -- open the vault, ask for master password, decrypt, do whatever you asked for, and if something changed, re-encrypt and save it back (see the Workflow Diagram in Section 7.2 for the full picture).
- **Bonus module:** a password generator (password_gen.py) that works even without an existing vault, just to quickly get a strong random password.

5. Non-Functional Requirements

Requirement	How I handled it
Security	Used PBKDF2-HMAC-SHA256 with 200,000 rounds and a random salt for both checking the password and generating the encryption key. Used Fernet (AES-128 + HMAC) for encryption so if someone messes with the file, it fails instead of silently giving corrupted data. Secrets never get printed in plain text or passed as a normal command-line argument.
Reliability	Every save writes to a temp file first, then swaps it in with <code>os.replace()</code> . So if the program crashes mid-save, your old vault file is still safe and not half-written/corrupted.
Usability	Password input is hidden while typing (<code>getpass</code>), secrets are masked when you view an entry unless you pass <code>--reveal</code> , there's a one-command clipboard copy option, and you can tag/search entries.
Performance	Everything runs in memory as a simple list, so even with hundreds or thousands of entries it's basically instant -- way more than a personal vault would ever realistically need.
Maintainability	Split into 7 separate files, each doing one job (auth, crypto, storage, entries, password gen, UI, CLI). Makes it a lot easier to test and debug each part on its own.

That's 5 non-functional requirements, one more than the minimum of 4 asked for.

6. System Architecture

I kept the CLI, the actual logic, the encryption, and the file-saving part all separate from each other. That way if I ever wanted to swap out how encryption works or how data gets stored, I wouldn't have to touch the CLI code at all. It also made testing each piece on its own a lot easier.

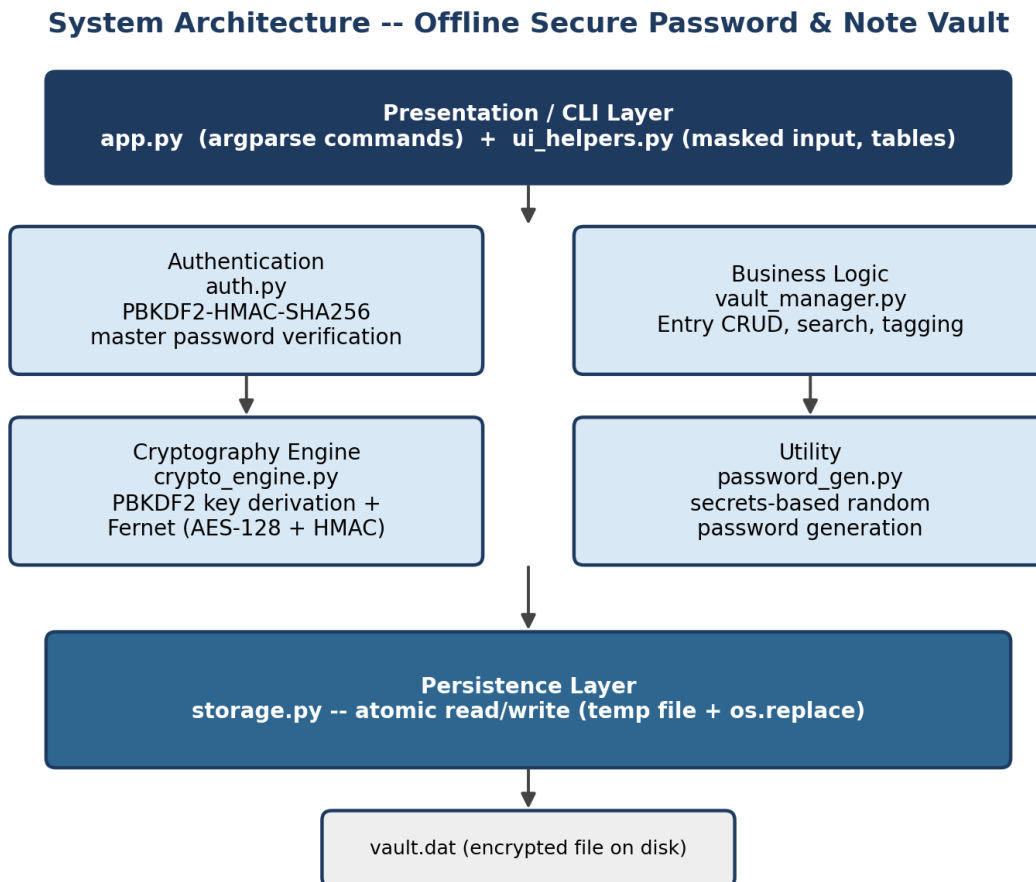


Figure 6.1 -- How the layers of the app fit together.

- **CLI Layer** -- reads your command and arguments, handles all the printing and masked password prompts.
- **Auth + Business Logic** -- checks your master password, does the actual add/edit/delete work on the decrypted data in memory.
- **Crypto + Utility Layer** -- derives keys, encrypts/decrypts, and generates random passwords.
- **Storage Layer** -- reads and writes the one encrypted file on disk, safely.

7. Design Diagrams

7.1 Use Case Diagram

There's really just one actor here -- the person using the vault. They can do 8 different things with it, and pretty much all of them (except generating a password) need you to authenticate first, since you can't touch any real data without unlocking the vault.

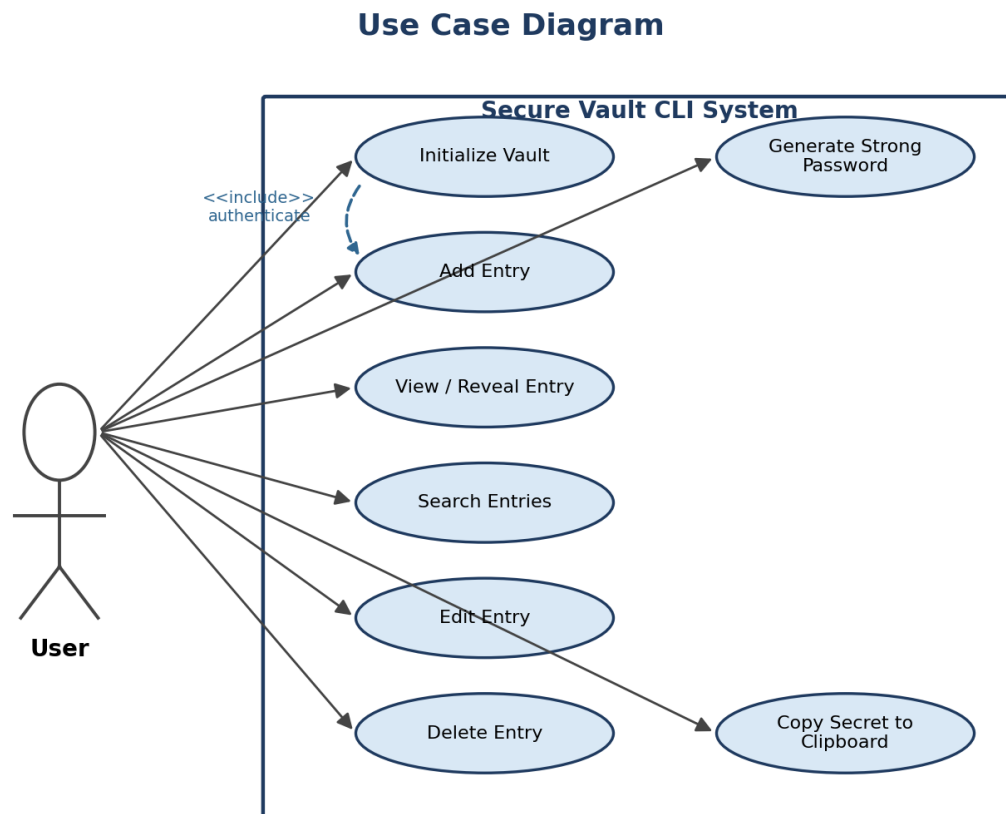


Figure 7.1 -- Use case diagram.

7.2 Workflow / Process Flow Diagram

This shows what happens, step by step, whenever you run any command: find the vault file, ask for your password, check it, decrypt everything, do the actual command, and then -- only if something actually changed -- re-encrypt and save before printing the result.

Process Workflow -- Running a Vault Command

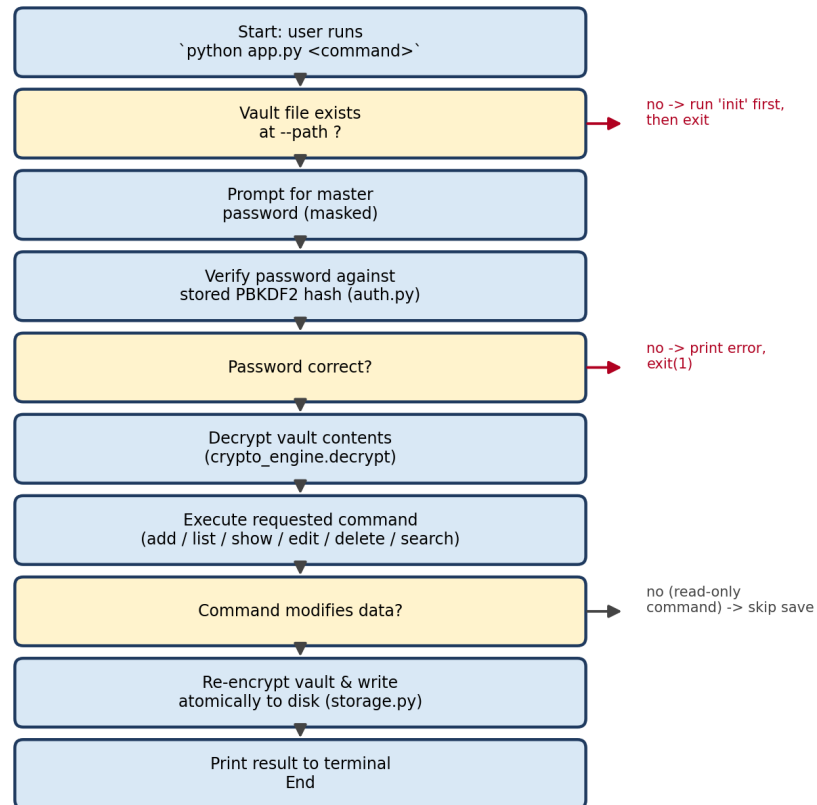


Figure 7.2 -- What happens when you run a command.

7.3 Sequence Diagram

This one traces through exactly what happens across all the files when you run the 'add entry' command -- from typing the command, to checking your password, decrypting, adding the new entry, re-encrypting, and finally saving to disk.

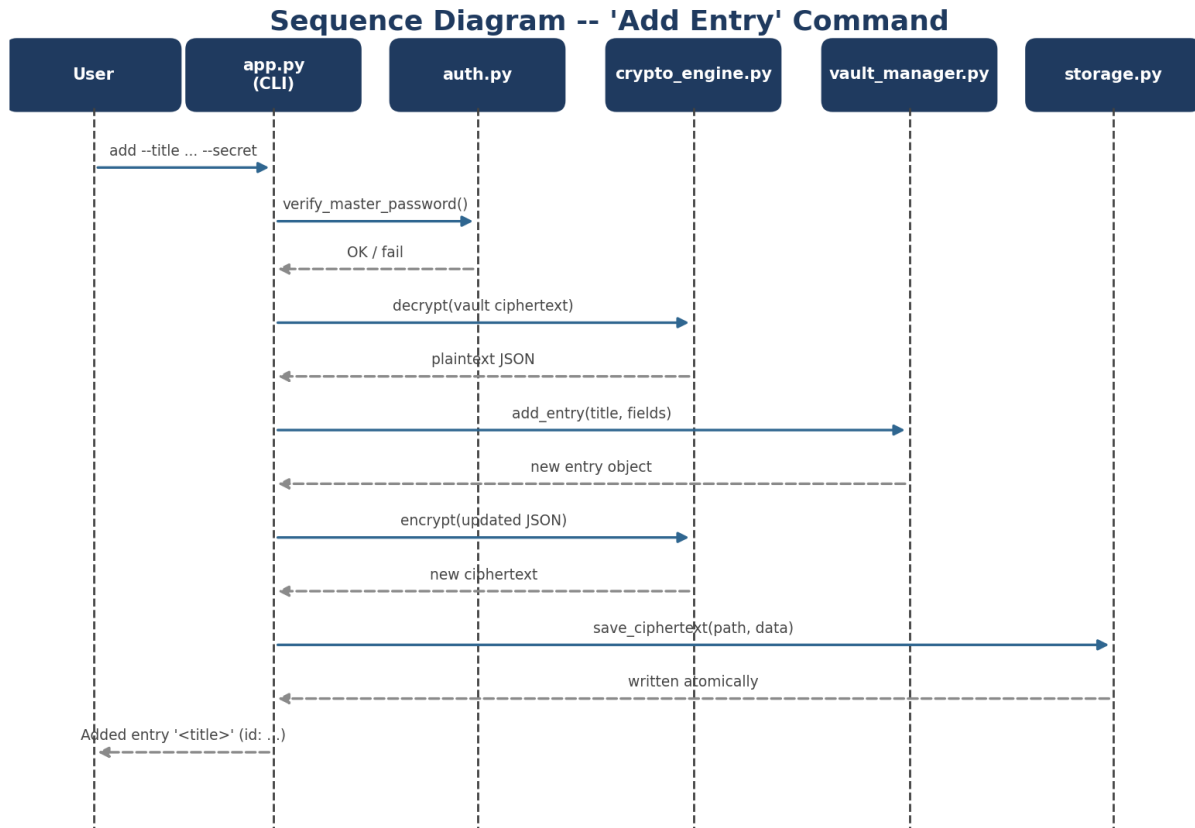


Figure 7.3 -- Sequence diagram for adding an entry.

7.4 Class / Component Diagram

This isn't a strict object-oriented program (it's mostly plain functions in modules), so I've drawn each module as a component showing its main functions instead of real classes. app.py is basically the "boss" module -- it calls into every other module.

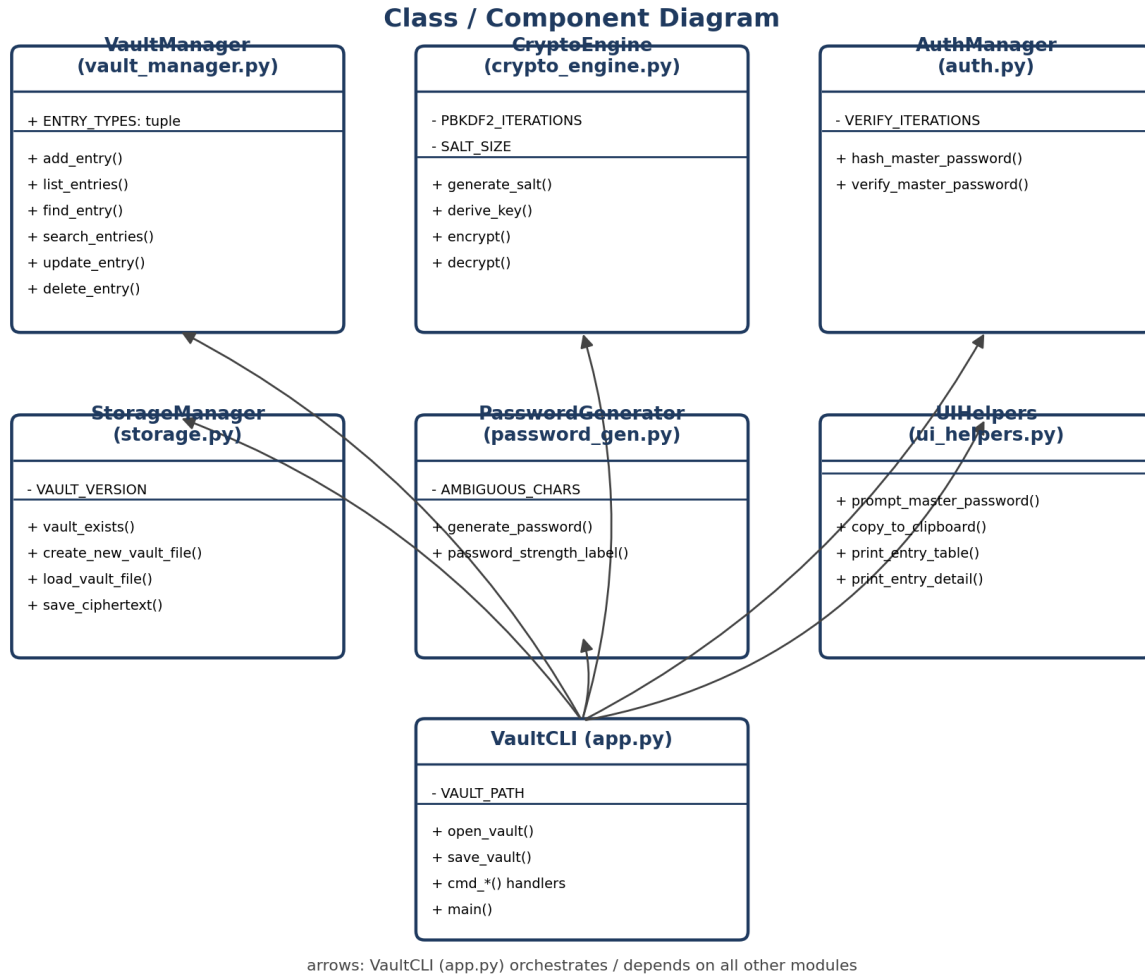
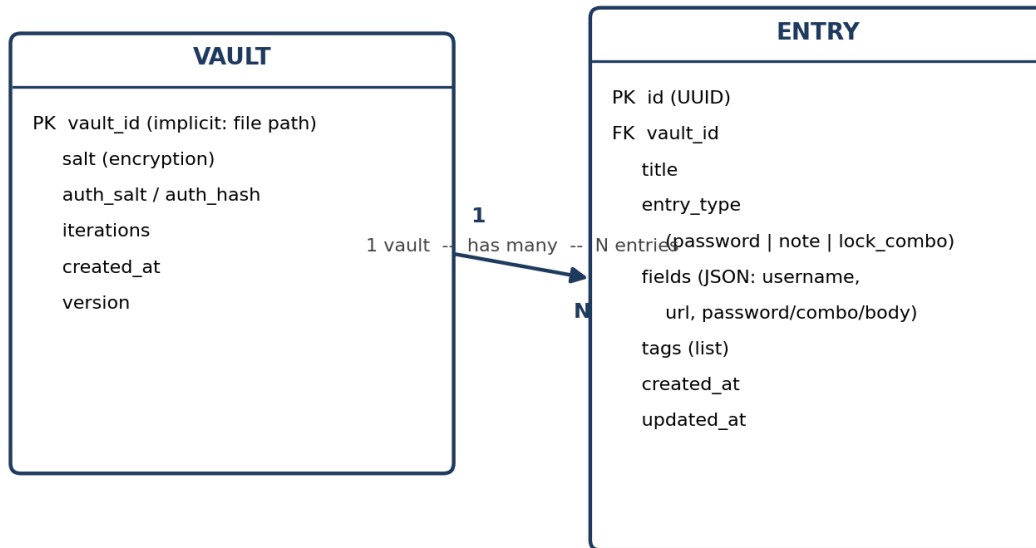


Figure 7.4 -- How the modules relate to each other.

7.5 ER Diagram

There's no actual database here -- all the entries just live inside one encrypted JSON blob per vault file. But logically it's still a one-to-many relationship (one vault has many entries), so I drew it out like a normal ER diagram to show that structure.

Entity-Relationship Diagram (Vault Storage Model)



Note: entries are stored inside one encrypted JSON blob per vault file (not a relational DB), but this ER model reflects the logical one-to-many relationship enforced in code.

Figure 7.5 -- The vault's data model.

8. Design Decisions & Rationale

8.1 Why Fernet instead of raw AES

I could've written raw AES encryption myself, but it's really easy to mess up (like reusing the same IV twice, or forgetting to check integrity). Instead I used the cryptography library's Fernet, which bundles AES-128 with an HMAC check built in. So if someone tampers with the vault file, decryption just fails loudly instead of quietly giving back garbage data.

8.2 Why two separate PBKDF2 keys

I derive two different things from the master password: one hash just to check if the password is correct (in auth.py), and a totally separate key used to actually encrypt/decrypt the data (in crypto_engine.py). Each has its own salt. This way, even if someone somehow got the verification hash, they still couldn't use it to decrypt your data.

8.3 Why atomic writes

If you write straight to vault.dat and the program crashes halfway through saving, you'd end up with a broken file and lose everything. So instead, storage.py writes to a temp file first, makes sure it's fully written to disk, and only then swaps it in place of the real file using os.replace() -- which is an atomic operation, meaning it either fully happens or doesn't happen at all. Your old vault is safe the whole time.

8.4 Why CLI-only, no GUI

The assignment says it has to run fully from the command line, so I didn't build any GUI or camera-based stuff. Masked input, clean tables, and clipboard copy give you decent usability without needing a GUI at all.

8.5 Why JSON instead of a real database

I went with one JSON file per vault instead of something like SQLite, mainly because it keeps things simple -- the whole vault gets decrypted and re-encrypted together as one unit, which is totally fine at the scale of a personal vault, and it's easy to read and debug.

9. Implementation Details

9.1 Tools Used

- **Language:** Python 3.9+
- **Main library:** cryptography (for PBKDF2 and Fernet)
- **Standard library stuff:** argparse, getpass, hashlib, hmac, json, os, tempfile, uuid, secrets, string, datetime
- **Optional:** pyperclip for clipboard copy -- if it's not installed the app still works fine, it just tells you clipboard isn't available
- **Testing:** Python's built-in unittest

9.2 Project Structure

```
vault-cli/  
|-- app.py           # CLI entry point (argparse commands)  
|-- auth.py          # Master password hashing & verification  
|-- crypto_engine.py # Symmetric encryption/decryption (Fernet + PBKDF2)  
|-- vault_manager.py # In-memory entry CRUD logic  
|-- password_gen.py  # Secure random password generator  
|-- storage.py        # Atomic on-disk read/write of the vault file  
|-- ui_helpers.py    # Masked input, table printing, clipboard  
|-- tests.py         # Unit tests (12 tests)  
|-- requirements.txt  
|-- .gitignore  
|-- statement.md  
`-- README.md
```

9.3 Codebase Size

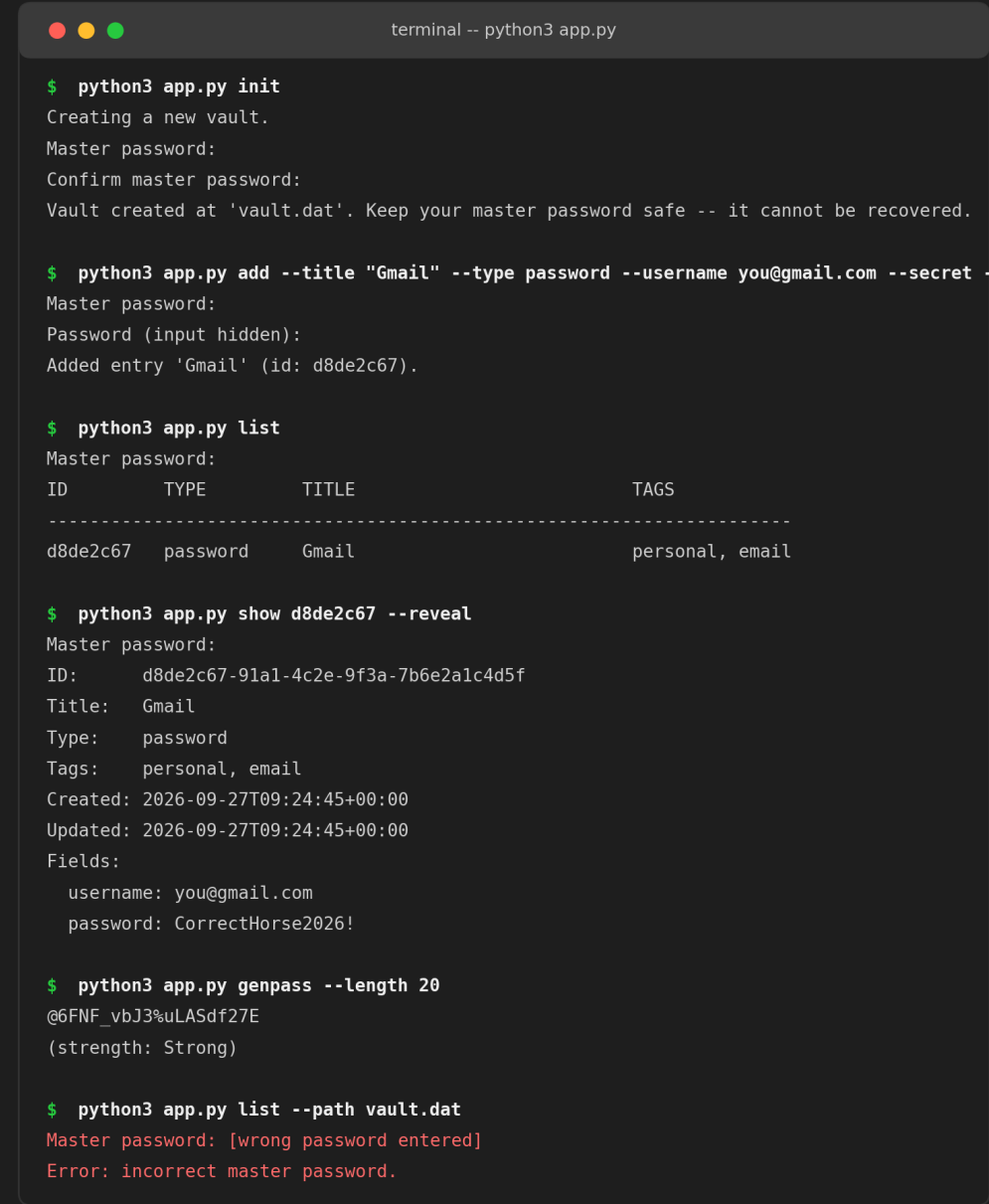
The whole thing is **8 Python files** (853 lines of code total, not counting docs), which fits comfortably in the 5-10 file range asked for, plus README.md and statement.md as required.

9.4 A snippet I'm proud of -- the atomic save function

```
def _atomic_write(path, data):  
    directory = os.path.dirname(os.path.abspath(path)) or "."  
    fd, tmp_path = tempfile.mkstemp(dir=directory, prefix=".vault_tmp-")  
    try:  
        with os.fdopen(fd, "w", encoding="utf-8") as f:  
            json.dump(data, f, indent=2)  
            f.flush()  
            os.fsync(f.fileno())  
        os.replace(tmp_path, path) # atomic on POSIX and Windows  
    except Exception:  
        if os.path.exists(tmp_path):  
            os.remove(tmp_path)  
        raise
```

10. Screenshots / Results

Here's a full run showing the app working end to end -- creating a vault, adding an entry, listing it, viewing it with the secret revealed, generating a random password, and then what happens if you type the wrong master password.



```
terminal -- python3 app.py

$ python3 app.py init
Creating a new vault.
Master password:
Confirm master password:
Vault created at 'vault.dat'. Keep your master password safe -- it cannot be recovered.

$ python3 app.py add --title "Gmail" --type password --username you@gmail.com --secret --tags personal,email
Master password:
Password (input hidden):
Added entry 'Gmail' (id: d8de2c67).

$ python3 app.py list
Master password:
ID          TYPE      TITLE      TAGS
-----
d8de2c67    password  Gmail      personal, email

$ python3 app.py show d8de2c67 --reveal
Master password:
ID:      d8de2c67-91a1-4c2e-9f3a-7b6e2a1c4d5f
Title:   Gmail
Type:    password
Tags:    personal, email
Created: 2026-09-27T09:24:45+00:00
Updated: 2026-09-27T09:24:45+00:00
Fields:
  username: you@gmail.com
  password: CorrectHorse2026!

$ python3 app.py genpass --length 20
@6FNF_vbJ3%uLASdf27E
(strength: Strong)

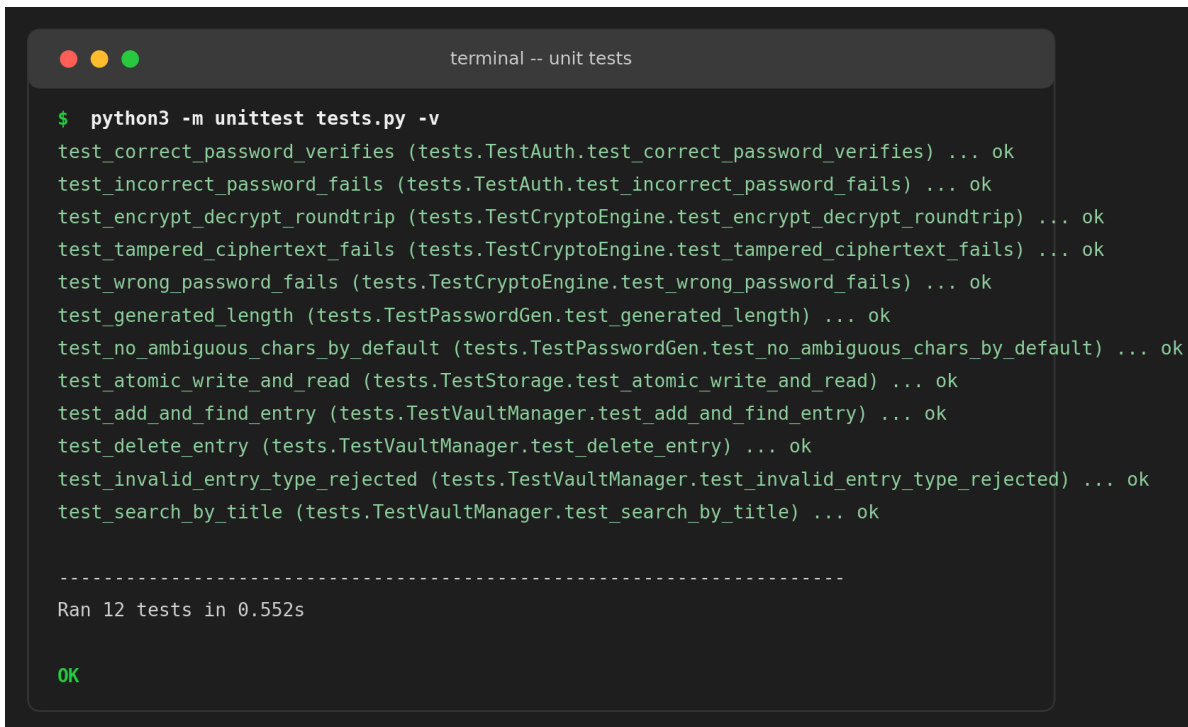
$ python3 app.py list --path vault.dat
Master password: [wrong password entered]
Error: incorrect master password.
```

Figure 10.1 -- Running the app from start to finish.

11. Testing Approach

I wrote unit tests (tests.py) using Python's built-in unittest, testing each piece of logic on its own, separate from the CLI:

- **Crypto:** encrypting then decrypting gives back the same data; wrong password gets rejected; a tampered file gets caught.
- **Auth:** correct password passes, wrong password fails.
- **Vault logic:** adding/finding entries works, bad entry types get rejected, search finds things by title, deleting actually removes the entry.
- **Password generator:** generated password is the right length, and ambiguous characters get excluded when asked.
- **Storage:** a file written to disk can be read back correctly.

A terminal window titled "terminal -- unit tests" with a dark background and light green text. It shows the command `$ python3 -m unittest tests.py -v` and the output of 12 tests, all passing with "... ok". The tests are: `test_correct_password_verifies`, `test_incorrect_password_fails`, `test_encrypt_decrypt_roundtrip`, `test_tampered_ciphertext_fails`, `test_wrong_password_fails`, `test_generated_length`, `test_no_ambiguous_chars_by_default`, `test_atomic_write_and_read`, `test_add_and_find_entry`, `test_delete_entry`, `test_invalid_entry_type_rejected`, and `test_search_by_title`. Below the list, it says "Ran 12 tests in 0.552s" and "OK" in green.

```
terminal -- unit tests

$ python3 -m unittest tests.py -v
test_correct_password_verifies (tests.TestAuth.test_correct_password_verifies) ... ok
test_incorrect_password_fails (tests.TestAuth.test_incorrect_password_fails) ... ok
test_encrypt_decrypt_roundtrip (tests.TestCryptoEngine.test_encrypt_decrypt_roundtrip) ... ok
test_tampered_ciphertext_fails (tests.TestCryptoEngine.test_tampered_ciphertext_fails) ... ok
test_wrong_password_fails (tests.TestCryptoEngine.test_wrong_password_fails) ... ok
test_generated_length (tests.TestPasswordGen.test_generated_length) ... ok
test_no_ambiguous_chars_by_default (tests.TestPasswordGen.test_no_ambiguous_chars_by_default) ... ok
test_atomic_write_and_read (tests.TestStorage.test_atomic_write_and_read) ... ok
test_add_and_find_entry (tests.TestVaultManager.test_add_and_find_entry) ... ok
test_delete_entry (tests.TestVaultManager.test_delete_entry) ... ok
test_invalid_entry_type_rejected (tests.TestVaultManager.test_invalid_entry_type_rejected) ... ok
test_search_by_title (tests.TestVaultManager.test_search_by_title) ... ok

-----
Ran 12 tests in 0.552s

OK
```

Figure 11.1 -- All 12 tests passing.

Other than the automated tests, I also just manually ran every command myself (init, add, list, show, search, edit, delete, genpass), with both the right and wrong master password, to make sure the errors show up properly instead of crashing (see Section 10).

12. Challenges Faced

- **Secrets leaking into shell history:** at first I let secret values get typed directly as command-line arguments, which would've ended up saved in shell history. Fixed it by adding a `--secret` flag that triggers a separate hidden prompt instead.
- **One key doing too much:** originally I used the same derived key both to check the password and to encrypt data. Realized that was risky, so I split it into two separate PBKDF2 derivations (see Section 8.2) so a leaked verification hash can't be used to decrypt anything.
- **Risk of corrupting the vault file:** my first version of `storage.py` wrote straight to `vault.dat`, which could corrupt it if the program crashed mid-save. Fixed by writing to a temp file first and swapping it in atomically (Section 8.3).
- **An actual bug my tests caught:** in the password generator, the `'avoid_ambiguous'` option filtered the main character pool, but not the pool used for guaranteeing one uppercase/lowercase/digit/symbol -- so characters like `'1'` or `'O'` could still sneak in even when I asked to avoid them. My test `test_no_ambiguous_chars_by_default` caught this directly, and I fixed it by filtering every pool consistently.

13. Learnings & Key Takeaways

- Got hands-on with key derivation (PBKDF2) and authenticated encryption (Fernet), and actually understood why "authenticated" matters -- it turns silent corruption into a loud, obvious failure instead.
- Learned why splitting code into small, single-job modules is worth it -- I could test auth, crypto, and storage completely separately from the CLI, which made debugging way easier.
- Understood why atomic file writes matter, and how something as simple as write-to-temp-then-rename avoids a whole category of "my file got corrupted" bugs.
- Saw firsthand that writing tests actually catches real bugs, not just imaginary ones -- the ambiguous character bug in Section 12 only got caught because I had a test written for that exact case.
- Got more comfortable building a CLI with argparse subcommands, closer to how real tools like git or docker are structured.

14. Future Enhancements

- **Encrypted export/import** so you can back up or move your vault to another machine.
- **Auto-lock after inactivity** so it re-asks for your password if you leave it sitting open too long.
- **Offline breach checking** to warn if a saved password shows up in a known leaked password list.
- **Multi-factor unlock**, like combining the master password with a key file stored locally.
- **A basic access log** (just timestamps, no sensitive data) to cover the logging/monitoring side more explicitly.

15. References

- 1 Python Software Foundation. *argparse -- Parser for command-line options, arguments and sub-commands*. Python 3 documentation.
- 2 Python Software Foundation. *getpass -- Portable password input*. Python 3 documentation.
- 3 Python Software Foundation. *secrets -- Generate secure random numbers for managing secrets*. Python 3 documentation.
- 4 Python Cryptographic Authority. *cryptography* library documentation -- Fernet (symmetric encryption) and PBKDF2HMAC (key derivation).
- 5 OWASP Foundation. *Password Storage Cheat Sheet* -- guidance on salting, iteration counts, and key derivation functions.
- 6 NIST Special Publication 800-132. *Recommendation for Password-Based Key Derivation*.
- 7 Python Software Foundation. *unittest -- Unit testing framework*. Python 3 documentation.